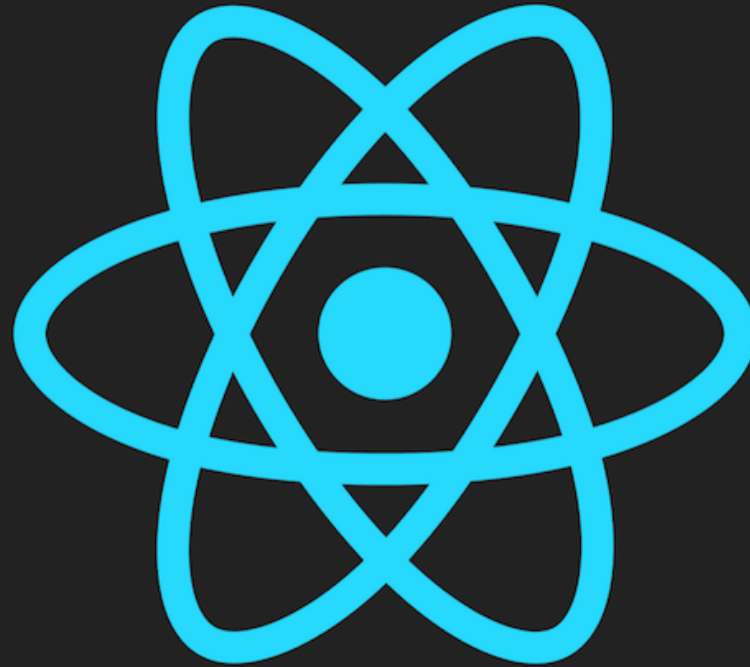




React Native

Componentes Básicos

View

- ▶ Container que suporta *layout* com flexbox, estilo, touch handling e controles de acessibilidade;
- ▶ Componente mais fundamental para construir a interface de usuário;
- ▶ Pode ser aninhado dentro de outras *views*.

View

```
import React from 'react';
import {View, Text} from 'react-native';

const ViewBoxesWithColorAndText = () => {
  return (
    <View
      style={{
        flexDirection: 'row',
        height: 100,
        padding: 20,
      }}>
      <View style={{backgroundColor: 'blue', flex: 0.3}} />
      <View style={{backgroundColor: 'red', flex: 0.5}} />
      <Text>Hello World!</Text>
    </View>
  );
};

export default ViewBoxesWithColorAndText;
```

Text

- ▶ Componente para exibir texto;
- ▶ Suporta aninhamento, estilização e touch handling.

Text

```
return (  
  <Text style={styles.baseText}>  
    <Text style={styles.titleText} onPress={onPressTitle}>  
      {titleText}  
      {'\n'}  
      {'\n'}  
    </Text>  
    <Text numberOfLines={5}>{bodyText}</Text>  
  </Text>  
)
```

Image

- ▶ Componente para exibir diferentes tipos de imagens;
- ▶ Permite incluir:
 - ▶ Imagens da Internet;
 - ▶ Recursos estáticos;
 - ▶ Imagens locais temporárias;
 - ▶ Imagens do disco local e da câmera.

```
<View style={styles.container}>  
  <Image  
    style={styles.tinyLogo}  
    source={require('@expo/snack-static/react-native-logo.png')}  
  />  
  <Image  
    style={styles.tinyLogo}  
    source={{  
      uri: 'https://reactnative.dev/img/tiny_logo.png',  
    }}  
  />  
  <Image  
    style={styles.logo}  
    source={{  
      uri:  
  
'data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAADMAAAAAzCAYAAAAAEXR  
FWHRTbZ20d2FyZQBwbmdjcnVzaEB1SfMAAABQSURBVGje7dSxCQBACARB+2/ab8BEeQNhfI6WS  
YzYLYudDQYGBgYGBgYGBgYGBgYGBgZmcvDqYGBgmhivGQYGBgYGBgYGBgYGBgYGBgbmQw+P/eM  
rC5UTVAAAAABJRU5ErkJggg==',  
    }}  
  />  
</View>
```


TextInput

- ▶ Componente para entrada de texto via teclado;
- ▶ Recursos suportados:
 - ▶ Autocorreção;
 - ▶ Autocapitalização;
 - ▶ Placeholder;
 - ▶ Teclado numérico, etc.

TextInput

```
const TextInputExample = () => {  
  const [text, onChangeText] = React.useState('Useless Text');  
  const [number, onChangeNumber] = React.useState('');  
  
  return (  
    <SafeAreaView>  
      <TextInput  
        style={styles.input}  
        onChangeText={onChangeText}  
        value={text}  
      />  
      <TextInput  
        style={styles.input}  
        onChangeText={onChangeNumber}  
        value={number}  
        placeholder="useless placeholder"  
        keyboardType="numeric"  
      />  
    </SafeAreaView>  
  );  
};
```

ScrollView

- ▶ Container de rolagem genérico que pode conter múltiplos componentes e views;
- ▶ A rolagem pode ser **horizontal ou vertical** (propriedade **horizontal**);
- ▶ Renderiza todos os elementos, mesmo os que não estão sendo vistos.

ScrollView

```
import React from 'react';
import {Image, ScrollView, Text} from 'react-native';

const logo = {
  uri: 'https://reactnative.dev/img/tiny_logo.png',
  width: 64,
  height: 64,
};

const App = () => (
  <ScrollView>
    <Text style={{fontSize: 96}}>What's the best</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 96}}>Framework around?</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 80}}>React Native</Text>
  </ScrollView>
);
```

StyleSheet

- Abstração similar ao CSS.

```
const App = () => (  
  <View style={styles.container}>  
    <Text style={styles.title}>React Native</Text>  
  </View>  
);  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    padding: 24,  
    backgroundColor: '#eaeaea',  
  },  
  title: {  
    marginTop: 16,  
    paddingVertical: 8,  
    borderWidth: 4,  
    borderColor: '#2032a',  
    borderRadius: 6,  
    backgroundColor: '#61dafb',  
    color: '#2032a',  
    textAlign: 'center',  
    fontSize: 30,  
    fontWeight: 'bold',  
  },  
});
```

Interface de Usuário

Button

- ▶ Insere um botão no App;
- ▶ Pode ser customizado;
- ▶ Você pode criar seu próprio botão utilizando Pressable.

```
<Button  
  onPress={onPressLearnMore}  
  title="Learn More"  
  color="#841584"  
  accessibilityLabel="Learn more about this purple button"  
/>
```


Switch

- ▶ Renderiza um input booleano;
- ▶ Requer uma função de *callback* **onValueChange** para atualizar o valor da propriedade que reflete a ação do usuário.

Switch

```
const App = () => {  
  const [isEnabled, setIsEnabled] = useState(false);  
  const toggleSwitch = () => setIsEnabled(previousState =>  
    !previousState);  
  
  return (  
    <View style={styles.container}>  
      <Switch  
        trackColor={{false: '#767577', true: '#81b0ff'}}  
        thumbColor={isEnabled ? '#f5dd4b' : '#f4f3f4'}  
        ios_backgroundColor="#3e3e3e"  
        onChange={toggleSwitch}  
        value={isEnabled}  
      />  
    </View>  
  );  
};
```

List Views

FlatList

- ▶ Recomendado para longas listas cuja **quantidade de itens muda** com o tempo;
- ▶ Renderiza apenas os elementos visíveis;
- ▶ Requer duas propriedades:
 - ▶ **data**: a lista
 - ▶ **renderItem**: recebe um item como entrada e retorna um componente formatado

FlatList

```
const FlatListBasics = () => {  
  return (  
    <View style={styles.container}>  
      <FlatList  
        data={[  
          {key: 'Devin'},  
          {key: 'Dan'},  
          {key: 'Dominic'},  
          {key: 'Jackson'},  
          {key: 'James'},  
          {key: 'Joel'},  
          {key: 'John'},  
          {key: 'Jillian'},  
          {key: 'Jimmy'},  
          {key: 'Julie'},  
        ]}  
        renderItem={({item}) => <Text style={styles.item}>{item.key}</Text>  
      }  
    />  
  </View>  
  );  
};
```

SectionList

- ▶ Para conjunto de dados divididos em seções lógicas.

```
const SectionListBasics = () => {
  return (
    <View style={styles.container}>
      <SectionList
        sections={[
          {title: 'D', data: ['Devin', 'Dan', 'Dominic']},
          {
            title: 'J',
            data: [
              'Jackson',
              'James',
              'Jillian',
              'Jimmy',
              'Joel',
              'John',
              'Julie',
            ],
          },
        ]}
        renderItem={({item}) => <Text style={styles.item}>{item}</Text>}
        renderSectionHeader={({section}) => (
          <Text style={styles.sectionHeader}>{section.title}</Text>
        )}
        keyExtractor={item => `basicListEntry-${item}`}
      />
    </View>
  );
};
```

Componentes Android e APIs

Componentes Android

- ▶ **BackHandler** – detecta o botão de hardware para navegação de retorno;
- ▶ **DrawerLayoutAndroid** – renderiza um DrawerLayout;
- ▶ **PermissionsAndroid** – provê acesso ao modelo de permissões do Android;
- ▶ **ToastAndroid** – cria um alerta toast.

Componentes iOS e APIs

Componentes iOS

- **ActionSheetIOS** – API para exibir uma action sheet iOS ou compartilhar sheet.

Obs.: action sheet é um modal view que apresenta escolhas relacionadas à ação do usuário.

Outros Componentes

Outros Componentes

- ▶ **ActivityIndicator** – exibe um indicador circular de carregamento;
- ▶ **Alert** – diálogo de alerta com título e mensagem;
- ▶ **Animated** – biblioteca para criar animações;
- ▶ **Dimensions** – interface para pegar as dimensões do dispositivo;
- ▶ **Linking** – interface para lidar com links;

Outros Componentes

- ▶ **Modal** – exibe conteúdo acima da view principal;
- ▶ **PixelRatio** – provê acesso à densidade de pixels do dispositivo;
- ▶ **RefreshControl** – usado dentro de um ScrollView para atualizar a tela;
- ▶ **StatusBar** – controla a barra de status do App.